SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

SupplyLah is a strategic solution designed to address critical inefficiencies in the Malaysian wholesale supply chain. Currently, Malaysian SMEs suffer from a manual bottleneck where they rely on unstructured, multilingual WhatsApp messages to manage daily operations. By replacing fragmented WhatsApp threads with a Stateful Multi-Agent Orchestrator, SupplyLah automates order ingestion, inventory reasoning, and logistics. The system is powered by ILMU-GLM-5.1, enabling it to understand mixed-language voice notes, text, and images to drive a seamless, end-to-end supply chain.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the development of SupplyLah. It focuses on how the system transitions from a linear script to an adaptive reasoning engine.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. SupplyLah is a cloud-native platform built on FastAPI. It utilizes three specialized GLM-powered units, which are Intake, Inventory, and Logistics Agents, all coordinated by a central stateful engine (orchestrator.py). The orchestrator manages the order lifecycle as a state machine starting with Pending, to Awaiting Confirmation, and finally Dispatched, utilizing a serialized inventory queue to prevent overselling.
2. ***Data Flows:***
    a. Inputs (text, voice, or photos) enter via Twilio/WhatsApp webhooks. Audio and images are processed via GLM Multimodal ASR/Vision and stored in AWS S3. The Intake Agent parses "bahasa rojak" against a product catalog fetched from Supabase. The Inventory Agent then cross-references stock and business rules. Once confirmed, the Logistics Agent triggers a (mocked) Lalamove API call, updating the Next.js Command Centre in real-time via Supabase subscriptions.
3. ***Model Process:***
    a. The model proceeds in four stages, which include ingest, involving multimodal mixed language capture using (ASR/Vision). Reasoning, where intake Agent extracts structured line items and confidence scores. Evaluate, where the Inventory Agent checks stock, applies pricing rules, and suggests substitutions. Execute where logistics agent orchestrates delivery and deducts inventory.
4. ***Role of Reference:***
    a. This document serves as the technical blueprint for the platform's evolution. it ensures the preservation of stateful memory and data contracts as the system moves from mocked services to full production integrations.

© UMHackathon 2026

1.2. Background:

For decades, the Malaysian wholesale supply chain has relied on high-velocity but low-structure communication via WhatsApp. Wholesalers are overburdened by shadow workflows, referring to manually reading 200+ messages, tracking stock on physical whiteboards or Excel, and manually booking couriers. This leads to massive cognitive load, frequent errors, and "inventory drift," especially during festive peak seasons.

1. *Previous Version:*
    a. The existing system is a manual, linear pathway limited to a single native WhatsApp application. It lacks centralized digital memory, making it impossible to scale and resulting in significant revenue loss from missed obligations and version chaos.
2. *Changes in Major Architectural Components*
    a. SupplyLah introduces a Non-Linear Agentic Reasoning Layer that fundamentally changes how orders are processed compared to traditional "Chatbot" scripts.
3. *New capabilities that are only introduced in this*
    a. **Multimodal Bahasa Rojak Reasoning:** The system uses GLM-5.1 to reason through mixed-language voice notes and photos of handwritten lists, capabilities impossible with standard keyword scripts.
    b. **Adaptive Substitution Logic:** The Inventory Agent doesn't just check stock, it reasons to offer intelligent alternatives when items are unavailable, maintaining the sales flow autonomously.
    c. **Stateful Orchestration:** Unlike simple bots, the orchestrator.py engine maintains the State of a conversation over time, allowing for multi-turn negotiations and proactive follow-ups if a customer stalls.
    d. **Self-Healing & Exception Handling:** The system detects low-confidence orders or process failures and automatically flags them for human review on the Command Centre Dashboard, ensuring high reliability in messy, real-world conditions.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Retail Buyer (Customer)	Orders supplies via WhatsApp using voice notes, text, or photos of order lists.	A seamless ordering experience in their preferred language , instant confirmations, and real-time delivery tracking.
SME Distributor/Wholesaler Admin (Owner)	Configures business or pricing rules, monitors the Command Centre, and	Increased order capacity without hiring more staff, reduced human errors in

© UMHackathon 2026

	handles high-level AI exceptions.	pricing/stock, and full visibility into automated operations.
Warehouse Manager	Monitors the fulfillment feed, prepares goods for dispatch, and confirms physical stock levels.	Structured, clear pick-lists generated by the AI and timely coordination with automatically booked logistics providers such as Lalamove.
Development Team	Builds, integrates, and maintains the multi-agent AI orchestrator and dashboard.	A modular codebase with independent agent boundaries, clear API contracts, and robust state management for the reasoning engine.
QA Team	Validates system behavior across multimodal inputs and tests edge-case handling.	Defined request or response formats for the GLM, consistent parsing of "bahasa rojak" inputs, and reliable failure recovery in API tool calls.
Strategic Partners (Ant International / Z.ai)	Provides the core AI engine (GLM-5.1) and evaluates ecosystem fit for SME digitalization.	Demonstrable proof that the GLM is the central reasoning engine and that the system solves high-friction trade issues for Malaysian SMEs.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*WhatsApp Conversation Interface & Web-based Dashboard (Next.js)*
Architecture	*Event-Driven Cloud Architecture with Multi-Agent Orchestration, structured as a single FastAPI backend that internally separates concerns into discrete agent pipelines (Intake, Inventory and Business Logic, Logistics), frontend deployed on Vercel and communication via REST.*

SupplyLah is structured as an event-driven, client-server system serving two distinct user endpoints: 1. Retail Buyers that interact purely via the Twilio WhatsApp API through conversations, and 2. Wholesaler Staff (Operators/Warehouse Managers) interact via a responsive Next.js web dashboard hosted on Vercel. Another user endpoint is SME distributors, which can act as both the buyers and sellers, in which here will be omitted for simplification purposes.

Both clients will communicate through a centralized FastAPI backend API layer deployed on AWS EC2. The backend operates on an asynchronous event loop, separating immediate webhook acknowledgements from heavy background processing tasks. Media storage (voice notes, images) is securely handled by AWS S3. All components share a unified Supabase ecosystem, utilizing PostgreSQL for relational order states and pgvector for storing knowledge base embeddings.

2.1.2 LLM as Service Layer

SupplyLah uses 3 distinct LLM models, with each assigned to a specific role in the pipeline based on their capability and cost profile. These models are invoked at distinct points in the workflow, each with a separately constructed prompt, a defined context window and a validated output contract.

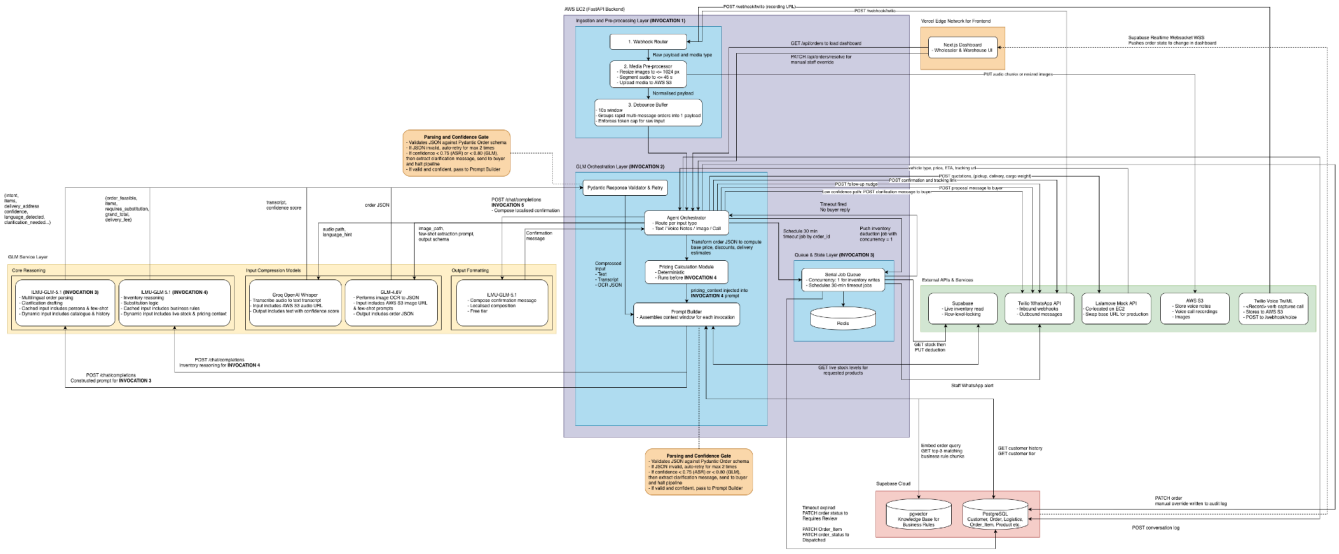
Task Assigned for Each Model

Model	Role	Point of Invocation
Groq OpenAI Whisper Large	Speech-to-text transcription	Voice notes and post-call recordings from AWS S3

© UMHackathon 2026

V3		
Z.ai GLM-4.6V	Multimodal OCR and entity extraction	Handwritten image and invoice orders from AWS S3
ILMU-GLM-5.1	Complex core reasoning, such as substitution, pricing, clarification drafting	All ambiguous, multilingual inputs, inventory logic, confirmation message composition

2.1.3 Dependency Diagram



Link: *DFD Diagram.png*

e. How are the prompts being constructed and sent to the GLM API?

SupplyLah does not send a single generic prompt to GLM, but uses the Prompt Builder module within the FastAPI backend to construct distinct, structured prompts for each GLM invocation points in the pipeline, tailoring models, tasks and runtime context.

The Prompt Builder passes the S3 pre-signed URL of the audio file directly as the input payload with language hint together with a perspective prompt to bias the transcription engine towards different Malaysian languages. This call is made via POST request to the transcription endpoint and returns a plain text transcript.

For GLM-4.6V, the Prompt Builder constructs a structured system prompt that tells the model to act as an order extraction specialist, where the target output JSON schema is defined, two to three few-shot examples of handwritten Malaysian

© UMHackathon 2026

wholesale order images with their correct structured extractions, and explicit instruction to mark any field that it cannot read with certainty as null rather than producing a guess. The image is passed as base64-encoded inline payload or via an S3 pre-signed URL in the same API call to chat completion endpoint.

For GLM-5.1 Invocation 3, it handles multilingual order parsing. The Prompt Builder assembles a multi-part system prompt consisting a role definition as a Malaysian wholesale intake specialist, the wholesaler's product catalogue retrieved from Supabase to resolve contextual references such as "sama macam hari itu", five to eight few-shot examples of Bahasa Rojak order messages paired with correct JSON extractions, the target output schema and conditional instructions for composing a clarification message in the buyer's detected language register if the confidence is insufficient. The static portions of the prompt, such as the role, few-shot examples and output schema are eligible for dynamic context caching, reducing per-session token cost.

For GLM-5.1 Invocation 4, which handles inventory reasoning and substitution logic, a separate prompt is constructed containing the wholesaler's business rules retrieved from Supabase via RAG vector search, the live stock levels for the requested products fetched directly from Supabase immediately before the call and a pre-computed pricing context produced by the deterministic Pricing Calculation Module that runs inside FastAPI before this GLM call is made. This design ensures that GLM is asked to reason over and validate a pre-computed price rather than perform arithmetic independently, which eliminates the known failure mode of GLM hallucinating numerical values.

For GLM-5.1 Invocation 5, the Prompt Builder constructs a lightweight prompt containing the confirmed order details, the logistics quote returned by the Logistics API and the buyer's detected language register from Invocation 3, with an instruction to compose a warm and concise WhatsApp confirmation message. All five prompts are sent as HTTPS POST requests to the respective GLM API endpoints from the FastAPI backend running on AWS EC2.

f. What goes into the context window?

The context window composition differs per GLM invocation and is managed explicitly by the Prompt Builder to stay within cost and latency targets.

For GLM-5.1 Invocation 3, the context window is structured in three tiers. The first tier is the cached system context, comprising approximately 800 tokens, which includes the system persona and role instructions, the few-shot Bahasa Rojak examples and the output JSON schema. These tokens are static across all buyer

© UMHackathon 2026

sessions for a given wholesaler operator and are billed at the reduced Z.ai cache-hit rate of USD 0.26 per million tokens rather than the standard input rate. The second tier is the dynamic context, comprising approximately 500 tokens billed at the standard rate, which includes the live product catalogue injected from Supabase and the buyer's last three order records retrieved from Supabase. The third tier is the live input, comprising 50 to 150 tokens depending on the input modality, which is the raw text message, the ASR transcript or the OCR-extracted JSON from the upstream compression model. The total context window for this invocation is approximately 1,350 to 1,450 tokens.

For GLM-5.1 Invocation 4, the cached portion contains the wholesaler's business rules retrieved via RAG from the Supabase pgvector Knowledge Base, comprising approximately 300 tokens. The dynamic portion contains the live stock levels from Supabase, the pre-computed pricing context from the Pricing Calculation Module and the validated order JSON, totalling approximately 350 tokens. The full context window for this invocation is approximately 650 to 700 tokens.

For GLM-5.1Invocation 5, the context window is approximately 300 tokens in total, comprising only the confirmed order summary, the logistics API response, and the language instruction. For Whisper and GLM-4.6V, context window management is handled differently, where ASR consumes audio tokens rather than text tokens, and the vision model consumes image tokens alongside the structured extraction prompt of approximately 400 tokens including few-shot examples.

g. How is your system receiving, parsing the responses and passing to the next system component?

All GLM API responses are received by the FastAPI backend as HTTPS JSON responses and immediately routed to the Pydantic Response Validator, which is the central parsing and handoff component in the system.

For ASR responses, the validator extracts the transcript string and a confidence score from the response metadata. If the confidence score falls below 0.75, the transcript is flagged as low quality and the pipeline is halted. The orchestrator instructs the Twilio API to send a clarification message to the buyer requesting them to re-record or type their order. If confidence is sufficient, the transcript string is passed to the Prompt Builder as the raw input for Invocation 3.

For GLM-4.6V vision responses, the validator parses the returned JSON object against the Pydantic Order schema and counts the number of fields marked as null by the model. If more than two fields are null, the confidence gate fails and the orchestrator sends a targeted clarification message to the buyer identifying which fields were unreadable. If the null count is within the threshold, the partial JSON is

© UMHackathon 2026

passed forward with the null fields flagged for clarification in the Invocation 3 prompt.

For GLM-5.1 Invocation 3 responses, the validator expects a JSON object containing order_json, confidence_score, detected_language, and an optional clarification_message field. The JSON is validated against the Pydantic Order schema. If the JSON is malformed or fails schema validation, the validator triggers an automatic retry with a refined prompt, up to a maximum of two retries. If retries are exhausted, the order is escalated to the staff dashboard. If the confidence score is below 0.80, the clarification_message field is extracted and dispatched to the buyer via the Twilio API and the pipeline halts until the buyer responds. If the confidence score meets the threshold and the schema is valid, the order_json is passed to the Agent Orchestrator, which triggers the Pricing Calculation Module and then the Prompt Builder for Invocation 4.

For GLM-5.1 Invocation 4 responses, the validator parses the fields can_fulfil, proposal_json, substitutions[], revised_total and proposal_message. The proposal_json is validated against the Order schema and a provisional order record is written to Supabase with the status PENDING_BUYER_APPROVAL. The proposal_message is dispatched to the buyer via the Twilio REST API and a 30-minute timeout job is pushed onto the serial queue keyed to the order session.

For GLM-4.7-Flash Invocation 5 responses, the validator extracts the plain text confirmation message string directly, which is sent to the buyer via Twilio. The order status is then updated to READY_TO_PACK in Supabase, then READY_FOR_DISPATCH after warehouse personnel packs, which triggers a Supabase Realtime WebSocket event that pushes the state change to the Next.js dashboard.

h. Where the limitation of token are enforced or input are chunked before it actually reaches to GLM

SupplyLah enforces all token and input size constraints within the FastAPI backend, in the Media Pre-processor and Debounce Buffer components, before any content is submitted to a GLM API endpoint. GLM models are never sent an unsegmented, oversized or unvalidated input.

For audio inputs, the Media Pre-processor splits any audio file longer than 60 seconds into overlapping 45-second segments, with a 3-second overlap at each boundary to prevent word truncation at segment edges. Each segment is uploaded to AWS S3 as a separate object, and each segment's S3 pre-signed URL is passed to Whisper individually. The transcripts are reassembled in order by the FastAPI

© UMHackathon 2026

backend after all segments are returned, before the merged transcript is passed to the Prompt Builder.

For image inputs, the Media Pre-processor uses PIL to resize all incoming images to a maximum resolution of 1024 by 1024 pixels before uploading to S3. Images that remain larger than 2 megabytes after resizing are rejected outright, and the buyer receives a clarification message asking them to resend a clearer or smaller photograph. This resize step is enforced before the S3 upload, meaning the GLM-4.6V model never receives a full-resolution image.

For text inputs, the Debounce Buffer enforces a 500-token hard cap on the raw_input field that is injected into the Invocation 3 context window. Buyers frequently send orders as a sequence of short, rapid WhatsApp messages rather than a single composed message. The Debounce Buffer accumulates all messages received within a 5-second inactivity window and concatenates them into a single merged payload before forwarding to the Agent Orchestrator. The 5-second timer resets on each new incoming message and fires only after a 5-second gap, preventing partial orders from being parsed mid-composition. If the concatenated payload exceeds 500 tokens after merging, the buffer truncates to the most recent 500 tokens and flags the truncation in the orchestrator log.

For the inventory reasoning invocation, the RAG retrieval step is capped at the top three matching Knowledge Base chunks from pgvector, regardless of how many chunks exceed the similarity threshold, to prevent unbounded business rule injection expanding the context window beyond the target of 300 tokens for that invocation.

i. Also, do outline all the major API calls between the system components

The following describes all major API call relationships between the components of SupplyLah, following the request flow from buyer input to order completion.

Twilio calls the FastAPI backend at POST /webhook/whatsapp for all inbound WhatsApp messages from buyers, delivering the message payload including the sender's number, message type, message body, and any media attachment URLs. For voice calls, Twilio Voice calls the FastAPI backend at POST /webhook/voice after the call ends, delivering the S3 recording URL generated by the TwiML <Record> verb.

The FastAPI Media Pre-processor calls AWS S3 at PUT object to upload audio segments, call recordings, and resized images, and calls GET object to generate pre-signed URLs that are passed to the GLM models.

© UMHackathon 2026

The FastAPI Agent Orchestrator calls the Whisper API at POST /audio/transcriptions with the S3 audio URL for voice note and call transcription. It calls the Z.ai GLM-4.6V API at POST /chat/completions with the S3 image URL and structured extraction prompt for handwritten image parsing. It calls the Z.ai GLM-5.1 API at POST /chat/completions twice: once for Invocation 3 for multilingual order parsing and once for Invocation 4 for inventory reasoning and substitution logic. It calls the Z.ai GLM-4.7-Flash API at POST /chat/completions for Invocation 5 to compose the localised buyer confirmation message.

The FastAPI backend calls the Supabase REST API at POST /rest/v1/conversation_log to persist the raw input and parse JSON for every order. It calls GET /rest/v1/customer to retrieve the buyer's profile, tier, and order history for context injection. It calls POST /rest/v1/order to create a new order record and PATCH /rest/v1/order to update order status through each pipeline state transition. It calls the Supabase RPC endpoint POST /rest/v1/rpc/match_knowledge to perform the pgvector similarity search, passing an embedding of the current order JSON and receiving the top three matching business rule chunks from the Knowledge Base table.

The FastAPI backend calls the Lalamove Mock API at POST /quotations after order confirmation, passing the pickup address, delivery address, and estimated cargo weight, and receiving the vehicle type, price estimate, ETA, and tracking URL in response.

The FastAPI backend calls the Twilio REST API at POST /Messages to send all outbound WhatsApp messages to buyers, including pricing proposals, clarification requests, follow-up nudges, and order confirmations. It also calls POST /Messages to dispatch WhatsApp alert notifications to staff members for critical failures and timeouts. The FastAPI backend calls SendGrid via POST /mail/send to dispatch corresponding email alerts to staff.

The Next.js frontend calls the FastAPI backend at GET /api/orders to load the live order list on the dashboard, and at PATCH /api/orders/:id when a staff member performs a manual override of an order, which the backend validates and writes to Supabase with a full audit log entry.

j. ***Do show how frontend, backend, services, database and external API interacts with each other.***

The Next.js dashboard, hosted on Vercel's Edge Network, serves as the staff-facing operational interface and communicates with the FastAPI backend over HTTPS REST calls. It does not interact directly with Supabase, GLM, or any external API, all business logic is mediated by the FastAPI backend. For live order updates, the

© UMHackathon 2026

dashboard subscribes to Supabase Realtime over a WebSocket connection, which pushes order state change events directly to the frontend whenever the backend writes a status update to the PostgreSQL order table. This means the dashboard reflects order progression in real time without polling the FastAPI backend.

The FastAPI backend, deployed on AWS EC2, is the central coordination layer of the system. It receives all inbound events from Twilio via webhook, orchestrates the multi-agent GLM pipeline, manages order state in Supabase, and dispatches outbound messages via Twilio. The backend hosts the serial job queue in-process, backed by a co-located Redis instance, which serialises all inventory write operations and manages confirmation timeout jobs without requiring a separate queue service in the MVP deployment.

Supabase serves as the unified persistence layer for the system. The PostgreSQL database stores all structured data including customer profiles, order records, order items, logistics bookings, and conversation logs. The pgvector extension within Supabase stores the embeddings of the wholesaler's business rules in the Knowledge Base table and serves similarity search queries from the Prompt Builder during RAG retrieval. Supabase Realtime listens for row-level change events on the order table and broadcasts them to connected Next.js dashboard clients over WebSocket.

AWS S3, in the same region as the EC2 backend, stores all binary media assets generated during order processing. The FastAPI backend uploads files to S3 and generates pre-signed URLs that are passed directly to the Z.ai GLM APIs, meaning the GLM models fetch media assets from S3 without the binary content passing through the FastAPI process itself, which keeps backend memory usage low.

The Z.ai GLM APIs are consumed as external HTTPS services from the EC2 backend. Each of the five GLM invocation points in the pipeline makes an independent API call to the relevant Z.ai model endpoint. The models are stateless from Z.ai's perspective, all session context is assembled by the FastAPI Prompt Builder on each call and no conversational state is stored on the Z.ai side.

Twilio acts as the bidirectional WhatsApp communication gateway. Inbound messages from buyers are delivered to the FastAPI backend via webhook. Outbound messages to buyers and staff are dispatched from the FastAPI backend via the Twilio REST API. Twilio Voice additionally handles call interception via TwiML, recording the call audio and delivering the recording URL to the FastAPI voice webhook endpoint after the call ends.

k. *Must highlights the distributed components if the system is planning to be deployed across multiple services*

© UMHackathon 2026

SupplyLah is deployed across five distinct hosting environments, each chosen for the workload it serves and its proximity to the Malaysia-based user base.

The Next.js frontend is deployed on Vercel's Edge Network, which distributes the static build across Vercel's global CDN nodes. Malaysian users are served from the nearest edge location, minimising dashboard load latency. The frontend communicates with the FastAPI backend over HTTPS and with Supabase Realtime over WebSocket, both of which terminate at the ap-southeast-1 region.

The FastAPI backend, serial job queue, Redis instance, and co-located Lalamove Mock API all run on a single AWS EC2 instance in the ap-southeast-1 (Singapore) region. This region is chosen specifically to minimise round-trip latency to Supabase, AWS S3, and the Malaysian buyer base. In the MVP, serial queue and Redis are co-located on the same EC2 instance rather than deployed as a separate managed service, which simplifies the infrastructure footprint for the hackathon scope. The architecture is designed so that Redis can be migrated to AWS ElastiCache and BullMQ extracted to a separate worker process without changes to the application logic, as the queue interface is fully abstracted behind the BullMQ client.

Supabase is deployed on Supabase Cloud in the ap-southeast-1 region, co-located with the EC2 backend to minimise database call latency. Supabase manages the PostgreSQL instance, the pgvector extension, and the Realtime WebSocket server as a fully managed service, meaning no database infrastructure is operated directly by the team.

AWS S3 is provisioned in the ap-southeast-1 region and is accessed from both the EC2 backend (for uploads) and the Z.ai GLM APIs (for media retrieval via pre-signed URLs). By keeping S3 in the same AWS region as EC2, upload latency from the backend is minimised, and the pre-signed URL approach avoids routing binary media through the backend process when GLM models need to access files.

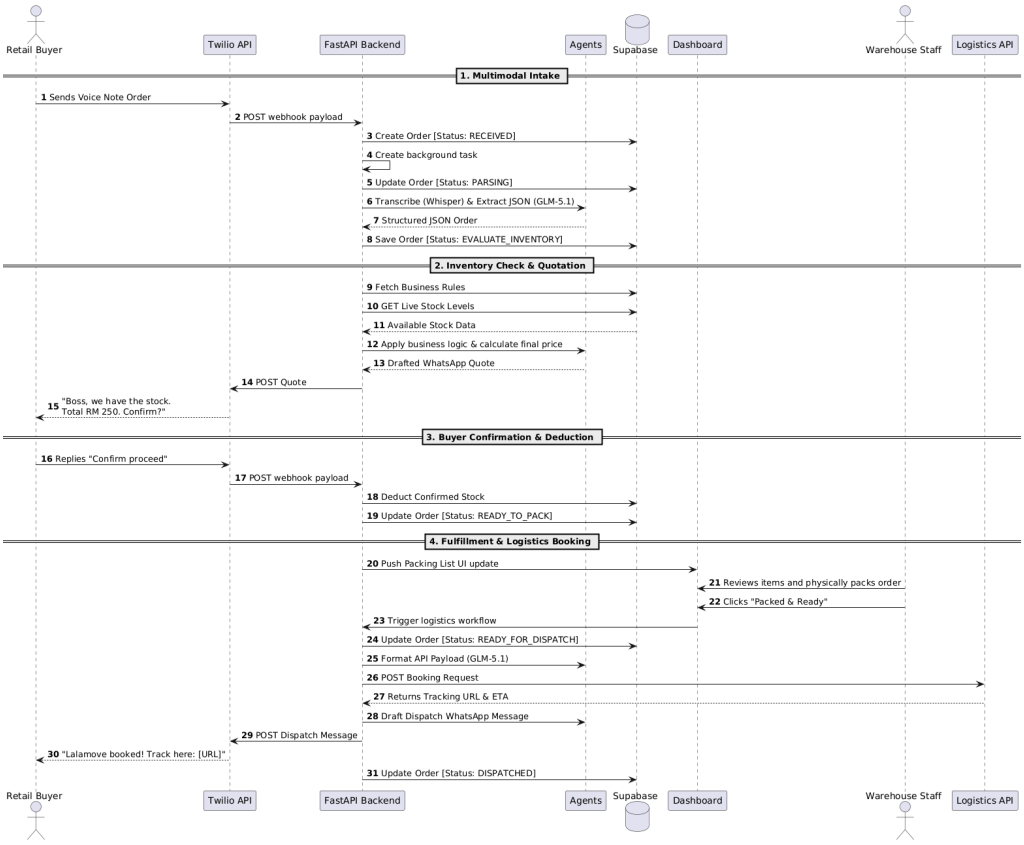
The GLM API endpoints are consumed as external cloud services from EC2 over outbound HTTPS. These are not deployed or managed by the team and represent the only distributed component outside the ap-southeast-1 region. All five GLM invocations originate from the EC2 instance, meaning GLM API latency is the primary external network variable in the system's end-to-end response time. Twilio's WhatsApp and Voice infrastructure similarly operates as an external cloud service, with inbound webhooks delivered to the EC2 public endpoint and outbound messages dispatched via the Twilio REST API.

2.2.2. Sequence Diagram

© UMHackathon 2026

Based on the various functionalities of SupplyLah, the following sequence diagrams describe several scenarios that the target users will be involved in, together with the hidden and user-facing modules that will lead from input to output generation.

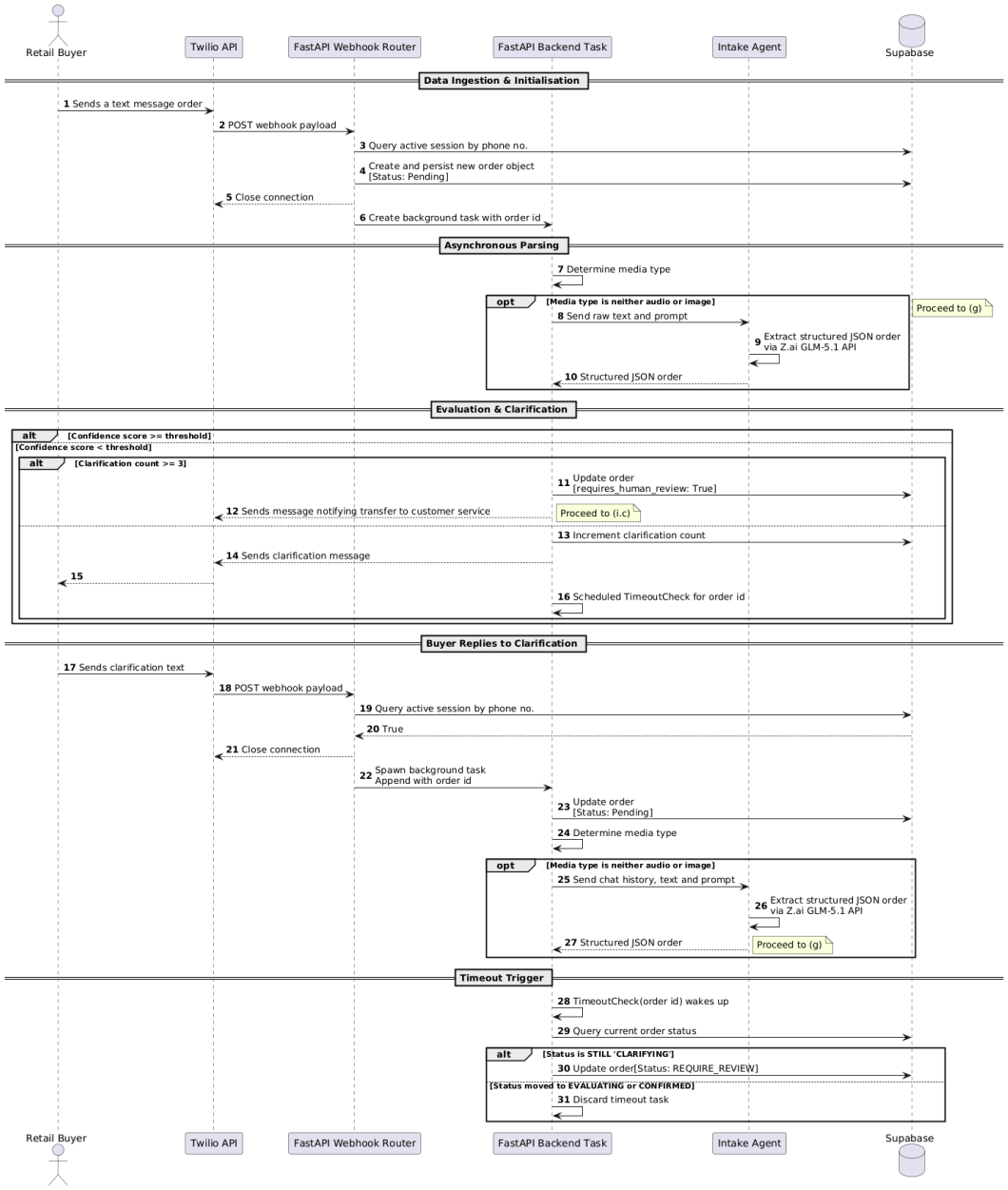
a. *Full End-to-End Interaction Between Buyer and Supplier*



The above diagram shows an outline of the end-to-end, seamless orchestration of SupplyLah platform from initial contact to physical delivery. When a retail buyer submits an unstructured voice note via WhatsApp, the FastAPI backend instantly transcribes the audio and structures the intent into a standardized JSON order using an intake agent, powered by Z.ai GLM-5.1 for core reasoning, OpenAI Whisper for speech-to-text transcriptions and GLM-4.6V for multimodal image input. Then, the logistics and business logic agent dynamically cross-referencing this JSON against the Supabase inventory and wholesaler specific rules to propose an accurate price quote. Upon buyer's affirmative response, the system safely deducts the inventory and passes it to the human warehouse staff via a real-time dashboard. Once physical packing is confirmed, the logistics agent autonomously books a delivery driver via third-party Logistics API and sends a final WhatsApp message with tracking details to the buyer. This complete cycle transforms messy, unstructured voice messages into a tracked, invoiced and shipped wholesale order process.

© UMHackathon 2026

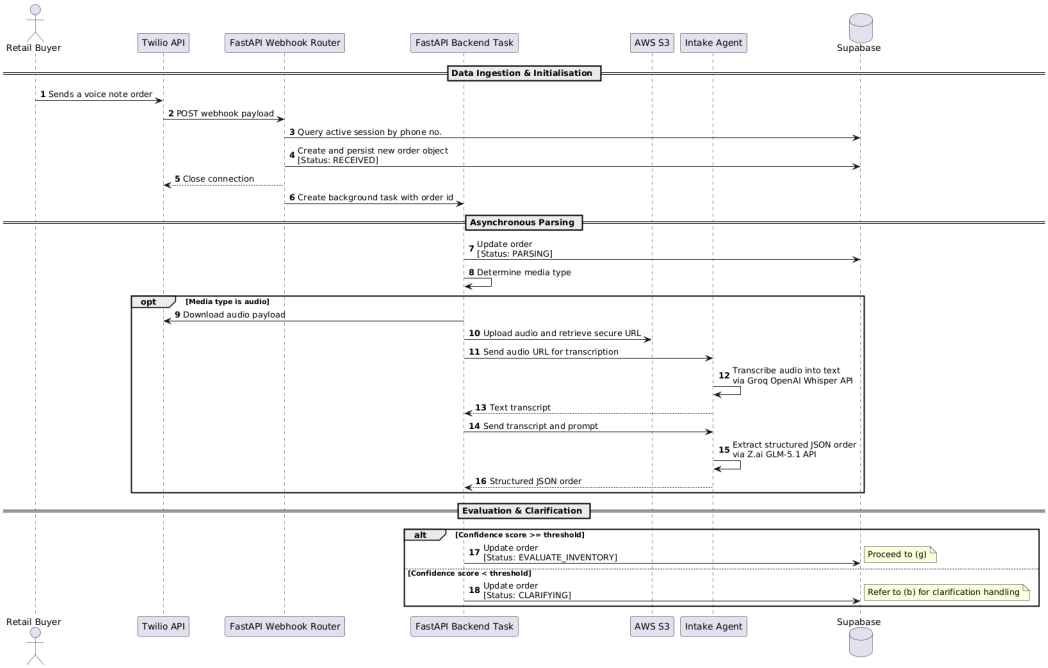
b. Text Message Order Intake



The above sequence diagram shows the base text message order ingestion process. When a buyer sends a text order via WhatsApp, the Twilio webhook will instantly be acknowledged by the FastAPI router to prevent timeouts, delegating parsing to an asynchronous background task. The intake agent uses GLM-5.1 API to extract a structured JSON order. If the AI confidence is low, it triggers a stateful clarification loop, asking the buyer for more details and waiting up to 30 minutes, for example, escalating to human review if it fails repeatedly or times out.

© UMHackathon 2026

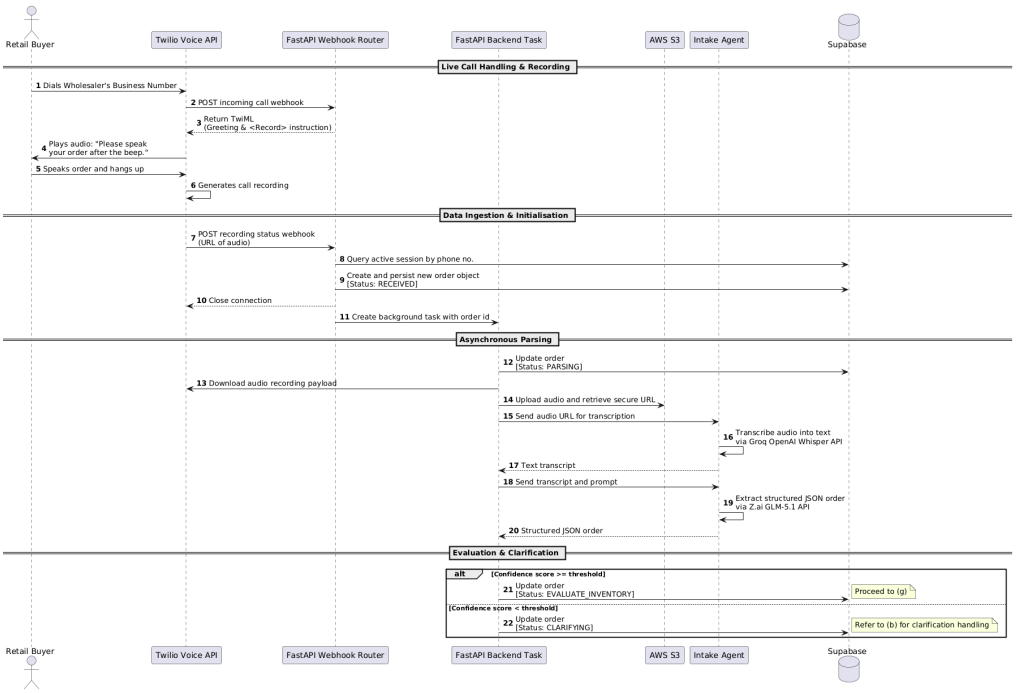
c. Voice Note Order Intake



This workflow handles unstructured audio messages. When receiving a voice note, the system securely uploads the media to AWS S3 and routes the file URL to Whisper for high accuracy speech-to-text transcription. The transcript is then processed by GLM-5.1 to extract the structured JSON order. Like the text flow, it incorporates a confidence score gate to automatically request clarification from the buyer if the transcribed intent is ambiguous.

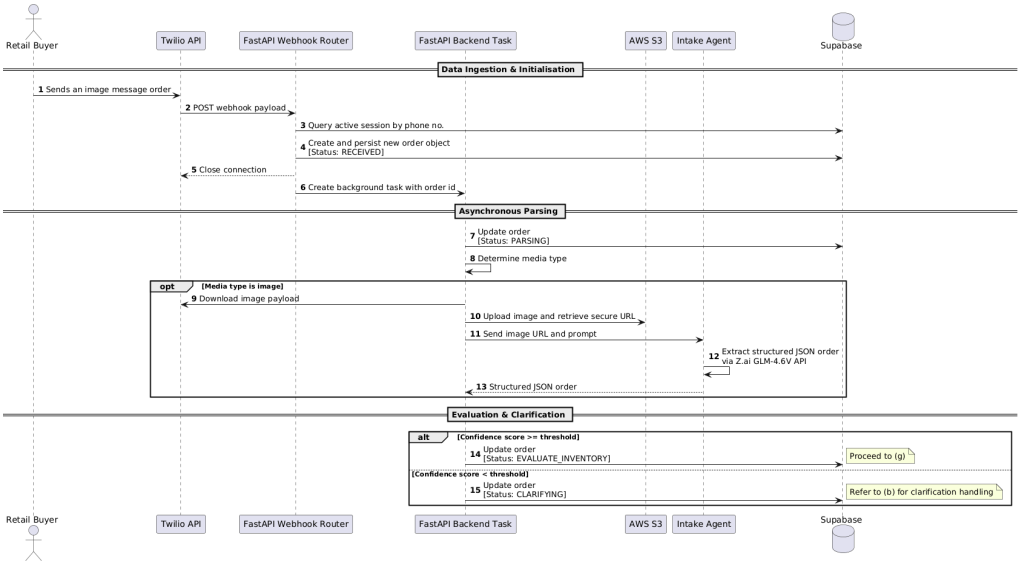
© UMHackathon 2026

d. Voice Call Order Intake



This workflow demonstrates a live phone call handling using Twilio Voice TwiML to record the buyer's spoken order. When the call ends, the recording is uploaded to AWS S3 and transcribed by Whisper, before GLM-5.1 structures the JSON. This flow bridges the communication channels, where the buyer places the order via a live voice call, but the AI continues the stateful interaction, such as clarifications or substitutions via asynchronous WhatsApp text messages.

e. Image Order Intake

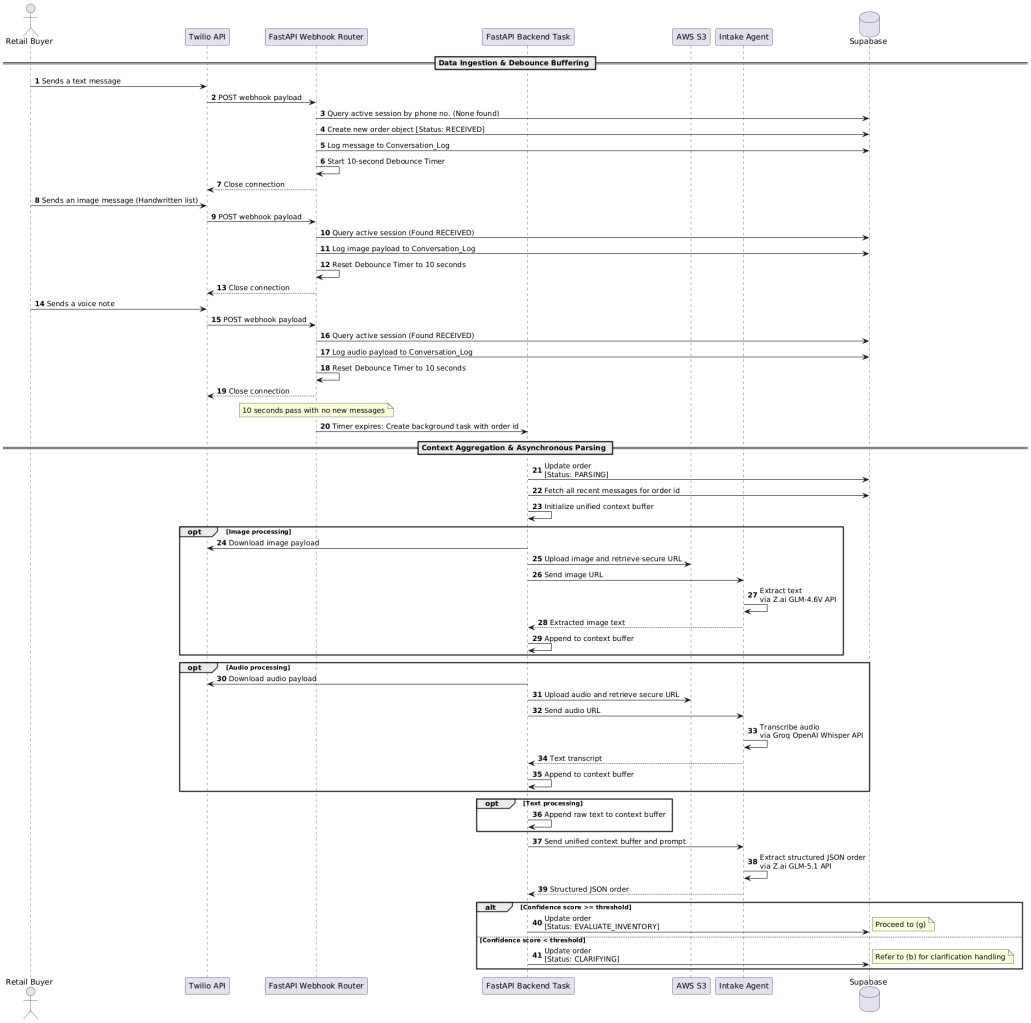


© UMHackathon 2026

Email: umhackathon@um.edu.my

For buyers who prefer sending photos of handwritten lists or invoices, this workflow securely stores the incoming images in AWS S3 before passing it to the multimodal GLM-4.6V Vision model. The GLM extracts texts and entities from the visual data and leads to GLM-5.1 to generate a structured JSON order. Low-confidence parses, commonly due to messy handwriting or blurry images, are safely caught by the clarification loop, ensuring inventory is only evaluated when the data extracted is reliable.

f. Mixed Input Order Intake

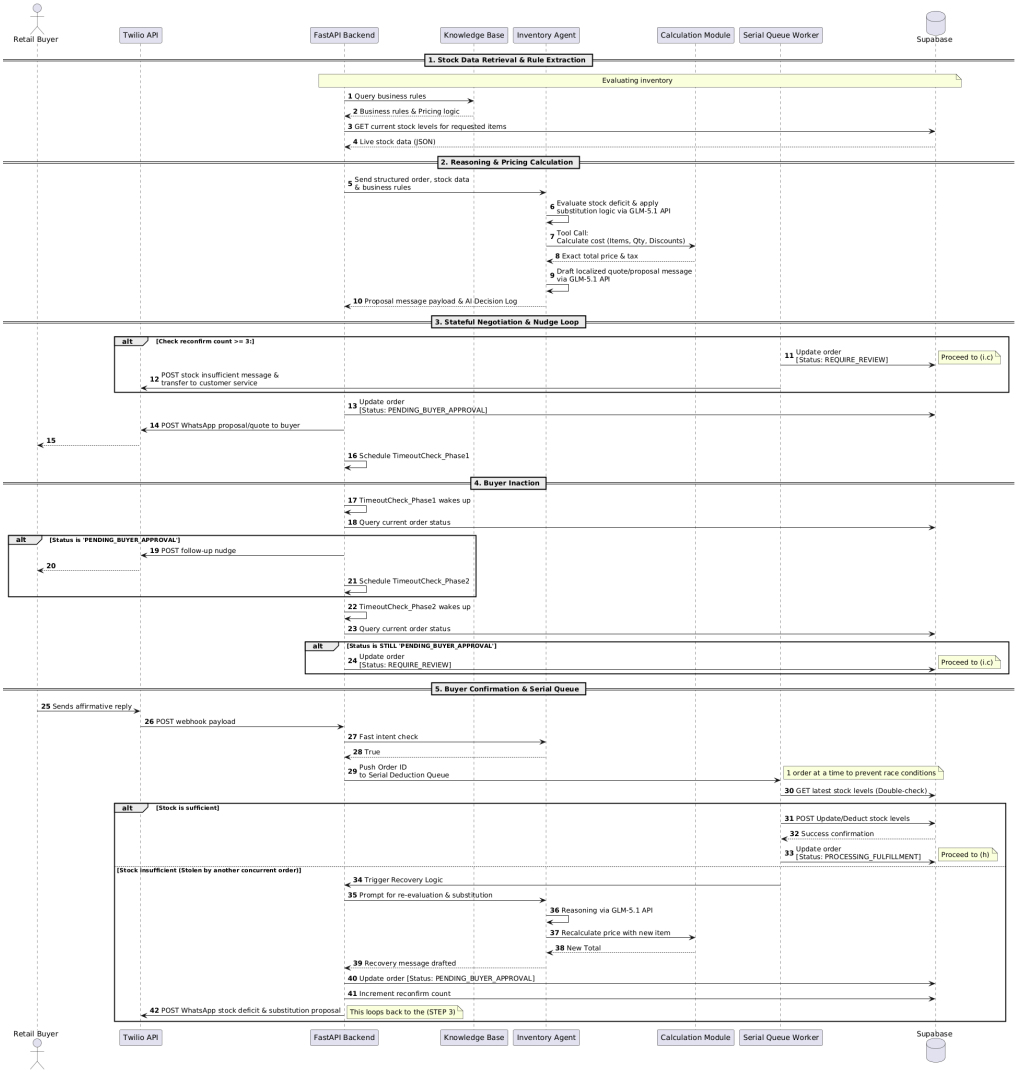


This diagram addresses the common issue of rapid-fire, mixed-media texting, such as a text, photo and a voice note. Instead of creating fragmented orders, the system uses a 10-second "Debounce Buffer" that resets with every new message. Once the buyer stops typing, the system aggregates all the text, transcribed audio (Whisper) and

© UMHackathon 2026

extracted image data (GLM-4.6V) into a single, unified context buffer. GLM-5.1 then evaluates all the entire context at once, improving accuracy.

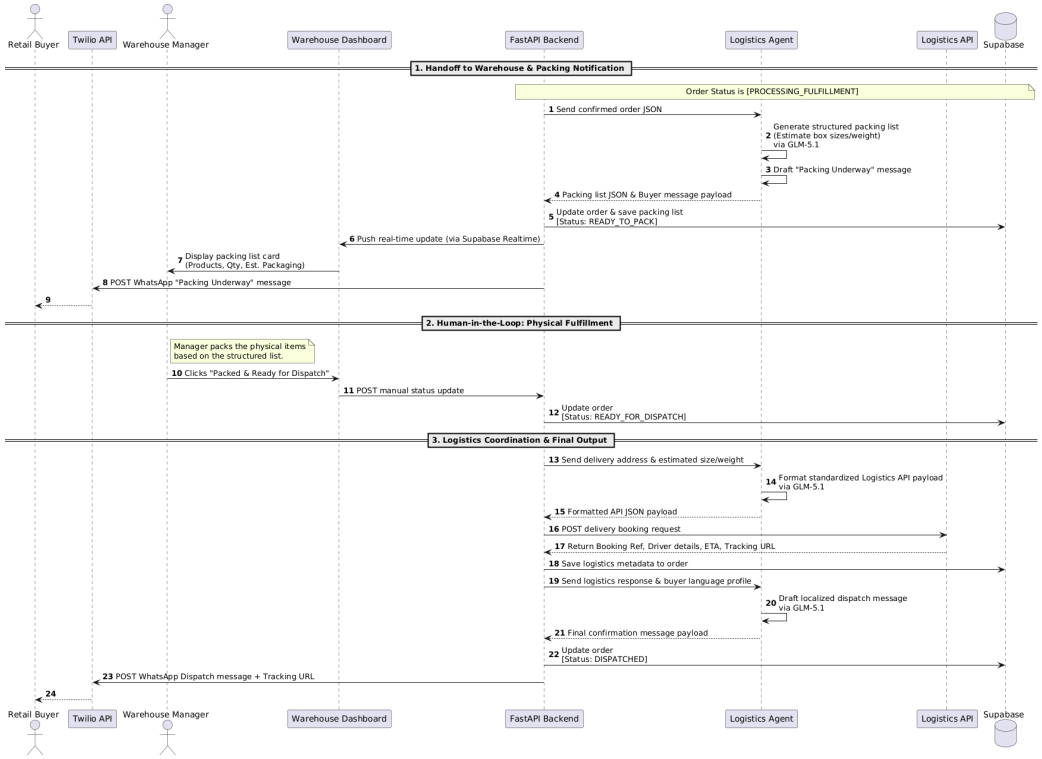
g. *Inventory Deduction and Serial Write Queue Mechanism*



The inventory and business logic agent queries the Supabase Knowledge Base via RAG for business rules and checks live inventory. GLM-5.1 reasons through stock deficits and negotiates substitute products with the buyer. Upon buyer confirmation, the order enters a Serial Queue which performs a double-check before stock deduction. If a race condition has occurred, the system intelligently triggers a recovery loop to propose new substitutes without crashing.

© UMHackathon 2026

h. Logistics Booking and Order Confirmation

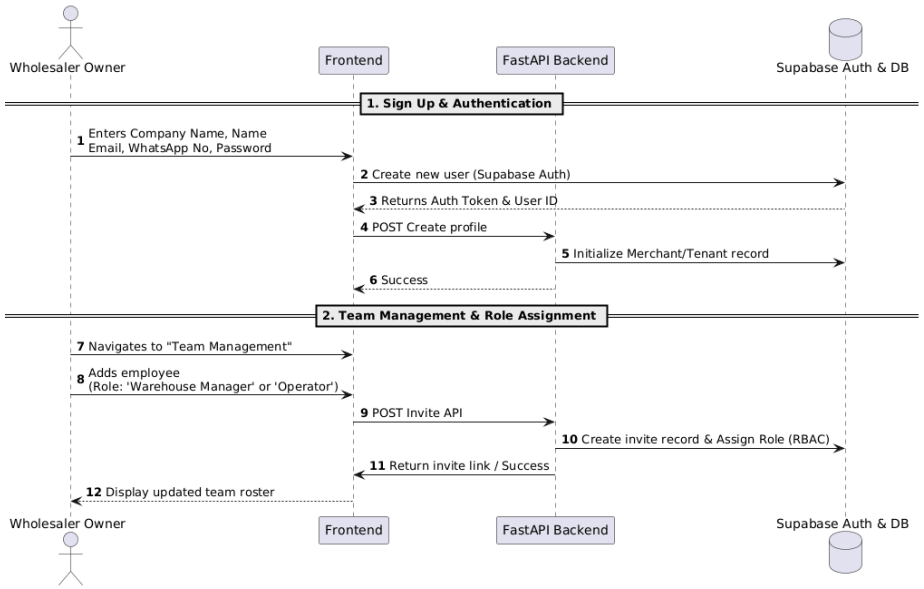


This workflow details the final physical handoff and external API integration using GLM-5.1 model. Upon receiving the confirmed order, the agent generates a structured packing list for the warehouse staff, such as estimating box sizes and groupings. Then it pushes to the warehouse manager dashboard. The system enters a stateful pause, waiting for the manager to physically pack the items, and responds with "Ready for Dispatch". Once triggered, the agent formats the cargo details and delivery address into a JSON payload to book a third-party delivery via Logistics API. Finally, it parses the API response to draft and sends a localized WhatsApp message to the buyer containing ETA and live tracking link, concluding the automated lifecycle.

© UMHackathon 2026

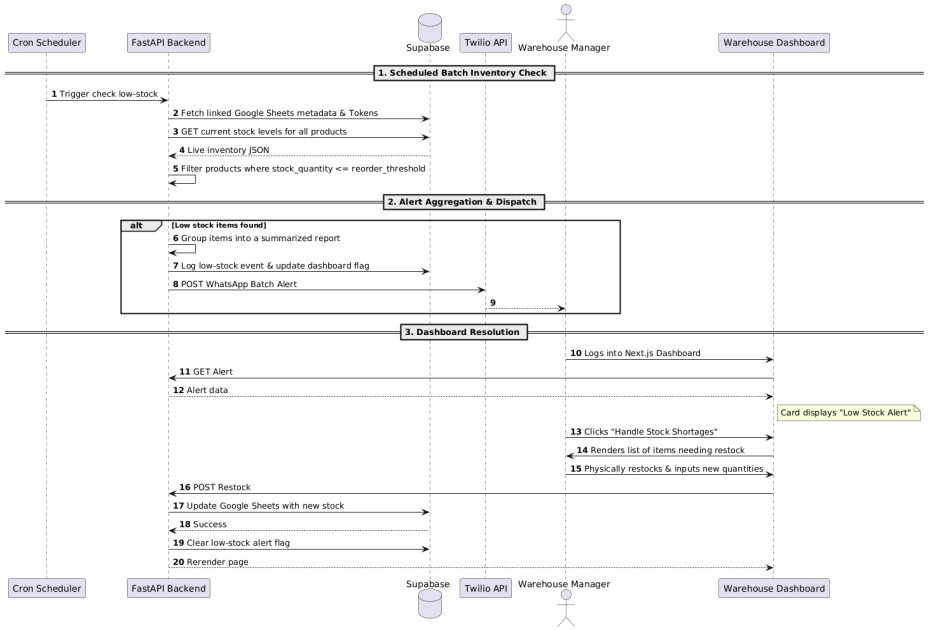
i. Auxiliary Modules

a. Wholesale Supplier Owner Onboarding



This workflow outlines the initialization for new wholesaler clients. It handles secure account creation via Supabase Auth. Finally, it demonstrates Role-Based Access Control (RBAC), allowing the owner to invite employees like Operators and Warehouse Managers to the platform with specific tiers.

b. Inventory Updates and Low-Stock Alerts

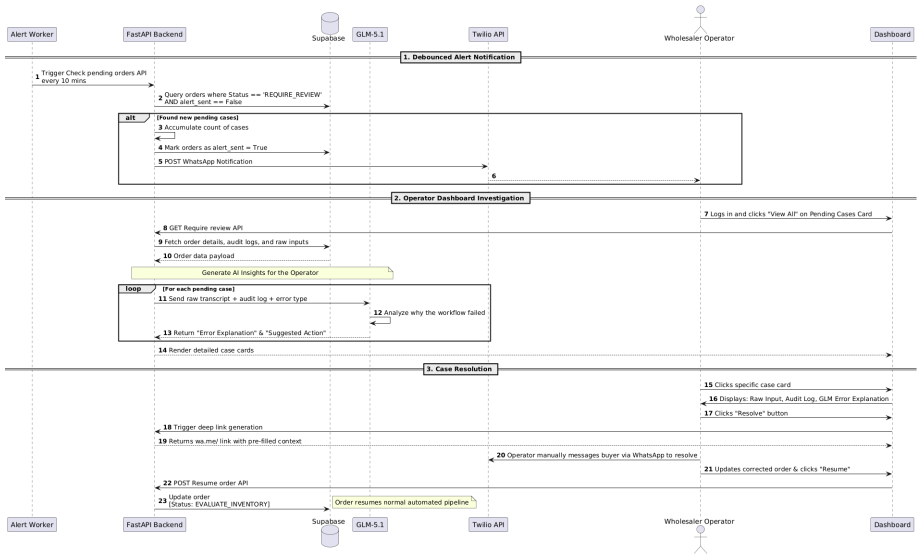


© UMHackathon 2026

Email: umhackathon@um.edu.my

To prevent notification fatigue, this diagram shows a scheduled cron job (e.g. once every 4 hours) to passively monitor the inventory instead of alerting the staff for every single deduction. When items drop below their reorder threshold, the system batches them into a single, summarized WhatsApp alert sent to the Warehouse Manager. With a dedicated action card on the dashboard, the manager can physically restock and update the database.

c. *Operator Handling Pending Alert Cases*



Link: *Pending.png*

This sequence diagram shows edge cases that require human intervention, indicated by the Requires Review status. A debounced worker will check for failed orders every 10 minutes (for example), batching notifications to avoid spamming the operator. To accelerate resolution, the system uses GLM-5.1 to analyse audit logs and provide the human operator with error explanation and suggested action. The operator resolves the issue directly and clicks "Review and Save" on the dashboard to push the order back into the automated pipeline.

2.2. Technological Stack

2.2.1. Frontend

Next.js – Responsive web-based Operational Dashboard for SME wholesalers and warehouse staff to monitor orders, manage inventory, and handle AI fallbacks

2.2.2. Backend

© UMHackathon 2026

FastAPI – RESTful API to handle Twilio WhatsApp webhooks, process multi-modal inputs, and orchestrate Z.ai GLM multi-agent workflows with MCP

2.2.3. Database

Supabase – Unified database for storing structured customer, order data and vector embeddings for business rule retrieval via RAG

2.2.4. Cloud/Deployment

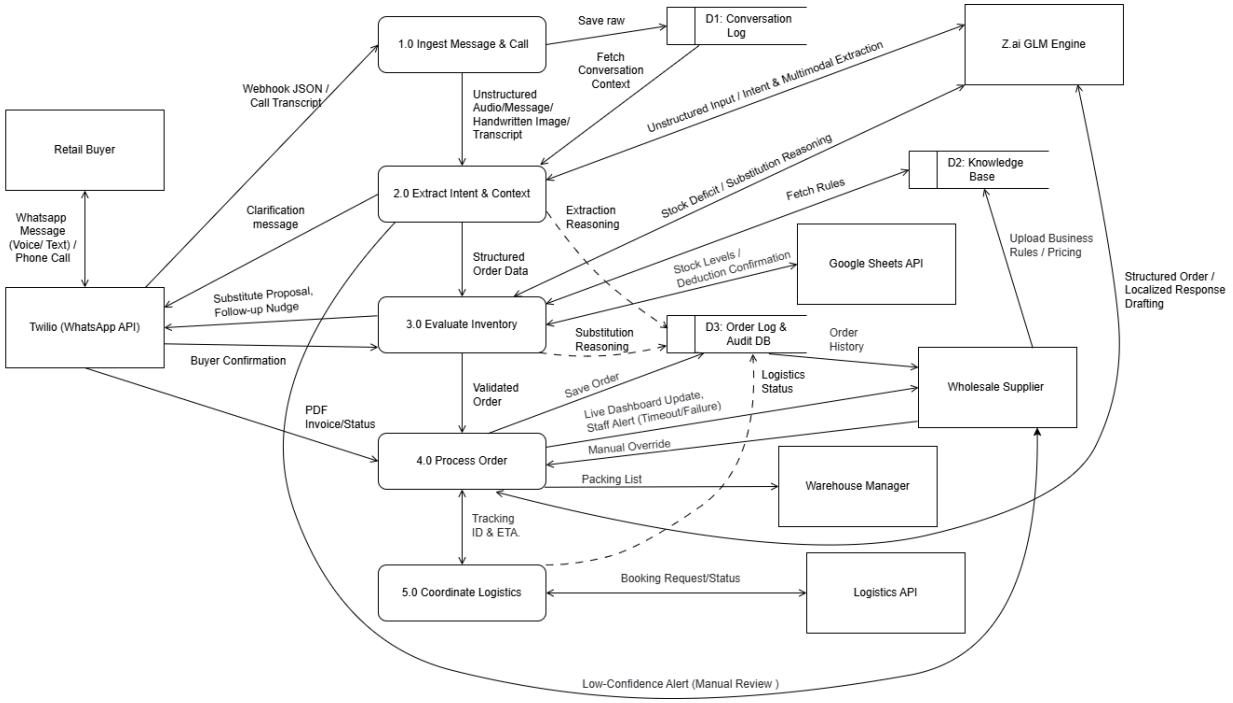
AWS EC2 (Backend Server), AWS S3 (Storage for WhatsApp voice notes,images), Vercel (Frontend Hosting), and Twilio API (WhatsApp Integration)

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

Link: [DFD Diagram.drawio]

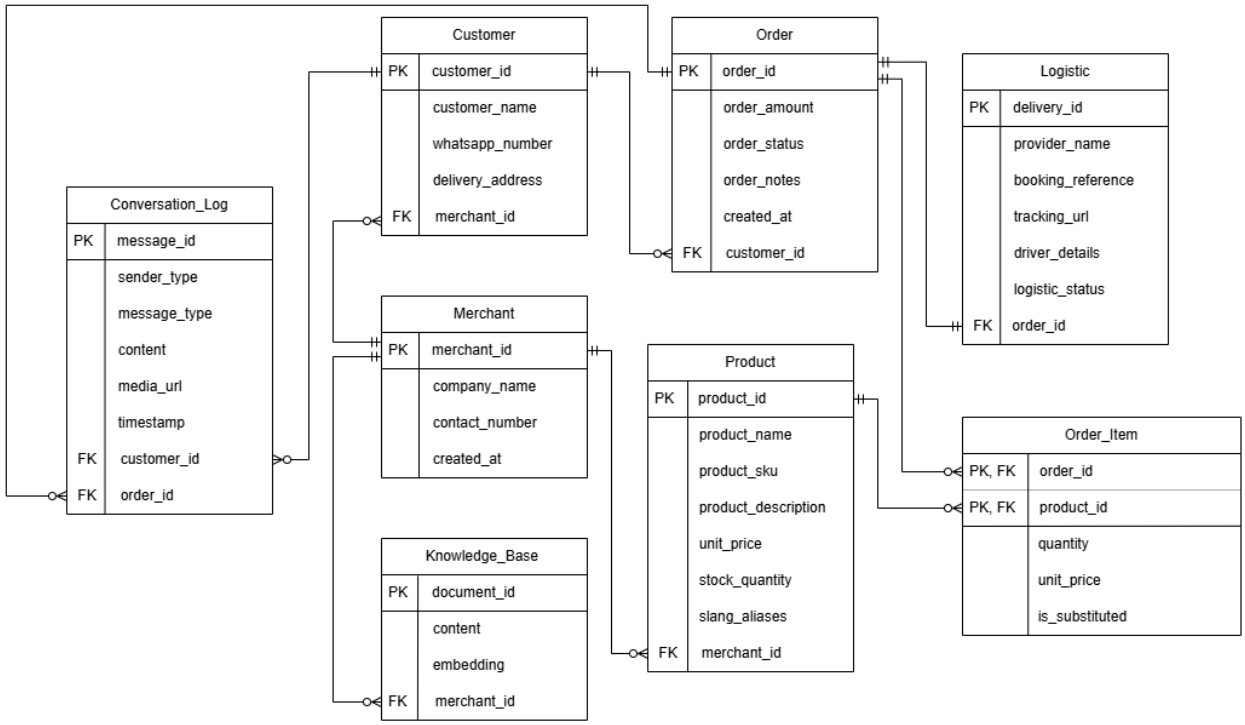


The SupplyLah DFD illustrates a stateful agentic architecture that transforms unstructured WhatsApp inputs from buyers to suppliers into structured logistics events while strictly adhering to data integrity laws. By ensuring every process and data store including the conversation logs and audit trails has both valid inputs and functional outputs, the system eliminates "Black Holes" and "Miracles." The Z.ai GLM Engine acts as the central brain, enabling real-time intent extraction and inventory negotiation, while the integration of external entities like Twilio and Logistics APIs ensures a transparent, end-to-end automation flow from the initial buyer message to final delivery confirmation.

2.3.2. Normalized Database Schema

Link: DFD Diagram.png

© UMHackathon 2026



SupplyLah database is architected in Third Normal Form (3NF) to ensure strict data integrity and eliminate redundancy across the wholesale supply chain. A central feature of this model is the Order_Item junction table. This junction table resolves the Many-to-Many (M:N) relationship between Order and Product using a composite primary key consisting of order_id and product_id. The schema further prevents update anomalies by decoupling Customer, Merchant, and Logistics entities, ensuring that delivery addresses or inventory changes are updated in a single location. The Conversation_Log entity helps to maintain stateful multimodal interaction history (text/audio), Knowledge_Base integrates an embedding vector column to support Retrieval-Augmented Generation (RAG).

3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. *Minimum Viable Product:*

3.1.1. *Here are 4 core features that we are going to build and do a demo by the end of the UMHackathon 2026.*

#	Features	Description
1	Multimodal Order Ingestion	Converts unstructured buyer messages (text, voice notes, calls, handwritten order images) via

© UMHackathon 2026

		WhatsApp into a structured JSON order using a pipeline of GLM models, acting as the intake agent.
2	*AI Inventory Reasoning and Negotiation*	*Cross-references parsed orders with live inventory and wholesaler business rules, GLM-5.1 autonomously calculates pricing, proposes stock substitutes and manages a stateful 30-minute confirmation loop with the buyer.*
3	*Automated Logistics Fulfillment*	*Upon order confirmation, the system will update the inventory via a database row-level locking and the logistics agent formats API payloads to book third-party logistics via mock approach, sending a localised WhatsApp tracking message to the buyer.*
4	*Wholesaler Operational Dashboard*	*A real-time Next.js web interface for warehouse manager staff to monitor active pipeline, view structured packing lists, update physical fulfillment statuses and manually resolve AI-escalated alerts.*

3.2. Non-Functional Requirements (NFRs)

These are several system qualities that ensure SupplyLah remains highly reliable, economically viable and stable during peak Malaysian wholesale operations.

Quality	***Requirements***	***Implementation***
Scalability	*The system must gracefully handle concurrent, rapid-fire messaging spikes during peak wholesale ordering hours without dropping webhooks.*	*The FastAPI router immediately returns HTTP 200 OK to Twilio webhooks, passing the heavy AI processing to an asynchronous background task queue (Redis/BullMQ) deployed on AWS EC2.*
Data Integrity via Concurrency	*The system must prevent the overselling of inventory stock when multiple buyers confirm orders simultaneously.*	*All inventory deductions are processed through a strictly serialized queue (concurrency: 1) that performs a live double-check of stock immediately before writing.*
Cost Efficiency	*The per-session AI inference cost must remain commercially viable for SMEs operating on thin margins.*	*Heavy media is compressed by Whisper and GLM-4.6V first, then complex reasoning uses GLM-5.1 with $0.26/1M static context caching for business rules, while simple formatting will use the free-tier GLM-4.7-Flash.*

© UMHackathon 2026

Performance (Latency)	*The system must accommodate human texting behaviour and prevent the creation of fragmented or duplicate orders from rapid, multi-message bursts.*	*A 10-second debounce buffer aggregates the sequential texts, voice notes and images from the same buyer into a single unified context window before triggering the reasoning engine.*
Maintainability	*The backend AI orchestration, database state and frontend UI must be loosely coupled to allow seamless updates without breaking active orders.*	*The Next.js UI interacts only with the FastAPI backend via REST and listens to Supabase Realtime via WebSockets. It never directly interacts with the GLM API.*

3.3. Out of Scope / Future Enhancements

The following high-level features are planned for the grand vision of SupplyLah to scale it into a complete enterprise SaaS, but are strictly excluded from UMHackathon 2026 MVP scope to maintain development focus on core agentic workflows.

a. ***Multi-Channel Support***
Expanding the Twilio integration to support other channels such as Telegram, Facebook Messenger and WeChat.

b. ***Dynamic Multi-Drop Logistics Management***
Implementing multi-stop route optimization for consolidating multiple single-point deliveries into efficient, single-trip logistics bookings.

c. ***Customer ERP & Accounting Integration***
Building bespoke API connectors and data synchronisation schemas for enterprise ERP and accounting software.

d. ***In-App Payment Collection***
Integrating licensed payment gateways for direct payment links within the WhatsApp conversation, which is subjected to Bank Negara Malaysia compliance.

e. ***Driver Tracking & Live Map Feature***
Building a native, high-frequency map rendering UI within SupplyLah's dashboard for live driver location states.

f. ***Full Business Intelligence (BI) Analytics***
Developing a comprehensive data warehouse pipeline with interactive charting for product-level margin tracking, delivery performance benchmarking and deep GLM reasoning audits.

g. ***Multi-Tenant SaaS Architecture***
Implementing automated self-service onboarding, tenant database isolation and subscription billing flows for multiple wholesaler organizations.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing

To ensure the stability of SupplyLah's automated workflows, we will utilize a feature flag based grayscale rollout for our MVP launch and all future backend updates. Rather than a risky all-at-once release, our core AI order processing pipeline like the GLM-5.1 reasoning engine and Whisper voice transcription will initially be deployed to a targeted 5% pilot group of active merchants. Our team will closely monitor the Operational Dashboard and backend logs for system health metrics, prioritizing the AI's intent extraction confidence scores and the frequency of "Red Alert" manual overrides from the different agent stages. If the system maintains a high accuracy rate over 24-48 hours and no issues or anomalies are detected, the rollout will gradually increase to 20%, 50%, 70% and eventually 100%. This strategy ensures heavy regional accents or highly complex "Bahasa Rojak" voice notes are caught and handled safely before they can disrupt automated workflows at scale.

Also, A/B testing is also conducted to optimize the prompt engineering on multi-agent architecture, specifically for the Intake Specialist Agent. We will test out different prompting strategies to determine the most effective way to extract key entities from unstructured and inconsistent "Bahasa Rojak" messages. For example, Variant A uses a concise few-shot prompt, injecting just 2-3 straightforward order examples to minimize input tokens and keep latency as low as possible. Meanwhile, Variant B employs a complex edge-case few-shot prompt, injecting 5-7 highly realistic examples that specifically tackle heavy slang, ambiguous quantities, and missing product names. We will evaluate these variants to determine if Variant B reduces "Low-Confidence" escalations enough to justify the higher token cost.

4.1.2. Emergency Rollback (ER) & Golden Release

For recovery and traffic management of the SupplyLah system, we implement a multi-layered emergency strategy. The first line of defense is a "Feature Switch" (Feature Toggle) system, allowing us to instantly turn off specific high-risk modules, such as Live Voice Call Transcription or Handwritten Image Parsing if they begin producing a spike in "Red Alert" manual overrides. By flipping these switches, we can isolate a faulty component and revert that specific workflow to a manual staff-review process without needing to shut down or roll back the entire platform, ensuring that standard WhatsApp text orders remain functional.

If toggling individual features fails to resolve system-wide instability, or if critical anomalies like repeated Google Sheets API write-back failures are detected, an Emergency Rollback (ER) is automatically triggered. The automated deployment pipeline immediately reverts to and

© UMHackathon 2026

deploys the Golden Version of the application. This Golden Version is the last certified, stable configuration of our multi-agent reasoning logic and few-shot prompt templates that have been proven to maintain our 90-second end-to-end processing target and 100% inventory accuracy.

For handling inbound traffic in case of emergencies, SupplyLah utilizes a circuit-breaker pattern within its WhatsApp Business API integration. Instead of dropping inbound messages, all text orders and voice note transcripts are temporarily buffered in a persistent message queue to ensure no customer data is lost during the rollback. Also, the inventory serial write queue is temporarily paused to protect the integrity of the wholesaler's stock levels in Google Sheets. Once the Golden Version is successfully redeployed and health checks pass, the system chronologically processes the buffered orders and resumes automated inventory deductions.

4.1.3. Priority Matrix:

4.1.3.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]

Priority Level	*Trigger Conditions*	*Required Actions*
P1: Critical	*- AI intent extraction "Red Alert" rate > 20% across all channels.* *- Inventory failure where Google Sheets API write-back failure rate > 2%* *- End-to-end processing exceeds 180 seconds for > 15% of orders*	*1. Immediate Emergency Rollback (ER) by triggering the CI/CD pipeline to redeploy the Golden Version* *2. Instantly Pause the Inventory Serial Write Queue to prevent corrupted stock data in Google Sheets.* *3. Buffer all inbound WhatsApp messages in the persistent queue.* *4. Maintenance Mode is automatically activated on the WhatsApp Business API to inform buyers of a temporary processing delay while the system recovers.*
P2: High	*- Voice/Vision Processing Failure like Whisper or GLM-4.6V ' producing 0% confidence scores for 15 consecutive minutes.* *- Bahasa Rojak" misinterpretation rate > 15%* *- Lalamove/Grab API failing to return quotes for 3*	*1. Activate Feature Toggle like instantly disable the specific module that does not work as expected* *2. Revert affected workflows to Manual Staff Review via the Dashboard.* *3. Dispatch a high-priority notification to the wholesaler staff to alert them of the partial feature fallback.* *4. Flag any failing logistics API calls for manual price confirmation and delivery booking.*

© UMHackathon 2026

	consecutive timeouts (>15s)	
P3: Medium	*- Average processing time creeps between 90–120 seconds* *- WhatsApp message delivery status (DLR) takes > 20 seconds to update.* *- Wholesaler Dashboard lag or reporting delays*	*1. Review context caching performance.* *2. Batch non-critical updates like order logs to write every 5 minutes instead of instantly.* *3. Clear internal system cache files to resolve minor rendering or data-refresh lag on the Operational Dashboard.* *4. Check Z.ai GLM API usage logs to verify if latency is caused by model load or network transit issues.*
P4: Low	*- Minor alignment issues on the Operational Dashboard.* *- Daily order count summaries delayed by > 30 mins.* *- Historical search filters for orders older than six months taking > 5 seconds to return results.*	*1. Log in the development backlog for the next scheduled maintenance sprint.* *2. Perform a client-side hard refresh of the dashboard to clear temporary UI or notification glitches.* *3. Schedule a non-disruptive database index optimization for historical order tables during off-peak hours.* *4. Manually trigger any delayed report generation if specifically requested by the wholesaler.*

*4.2. **Monitoring**:*

4.2.1. Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.

All KPIs are captured via backend logging pipelines and monitored through the Operational Dashboard, with automated threshold-based triggers driving alerting and recovery actions. The monitoring rules and corresponding system responses are summarized in the table below

Monitoring Layer	*Key Metric (KPI)*	*Alert/Trigger Threshold*	*Action Taken*
AI Reasoning	*Intent Extraction Confidence Score*	*Average confidence score < 0.70 for 3 consecutive orders or >15% within a 10-minute window*	*P2: Flag as "Low Confidence" and route to Manual Staff Review.*
Processing Speed	*End-to-End Latency*	*> 120 seconds for 15% of traffic within a 15-minute window*	*P3: Log delay; optimize prompt tokens or switch to GLM-4.7-Flash.*

© UMHackathon 2026

Data Integrity	*Supabase Write-Back Success Rate*	*Failure rate >2% after retry attempts within a 60-minute window*	*P1: Trigger Emergency Rollback (ER) and pause Serial Write Queue.*
Logistics	*External API Response*	*> 15s timeout or 4xx/5xx errors for 3 consecutive requests*	*P2: Activate Feature Toggle to disable Auto-Booking and notify staff.*
System Throughput	*Order Queue Backlog Size*	*Queue length > 50 pending orders or processing delay > 5 minutes*	*P2: Slow down new incoming orders and increase processing capacity and notify staff*
User Experience	*"Red Alert" Override Rate*	*>20% of orders require manual intervention within a 30-minute window*	*P1: Redeploy "Golden Version" to restore reasoning stability.*

4.3. ***Assumptions:*** *State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.*

a. *Wholesalers and warehouse managers have access to a stable internet connection to access the web-based Operational Dashboard.*
b. *The target buyer has WhatsApp installed and is familiar with sending voice notes and images as their primary communication tool to make orders.*
c. *The wholesaler maintains their Google Sheet inventory according to the fixed schema provided to ensure the agent's Function Calling and API read or writes execute without schema mismatch errors.*
d. *Buyers' mobile devices are capable of recording clear audio for Whisper processing, and cameras are sufficient to capture legible photos of handwritten lists.*
e. *The "Bahasa Rojak" used follows common Malaysian linguistic patterns which the GLM-5.1 model has been few-shot trained to interpret.*
f. *When buyers provide a delivery address via text or voice, the address is complete and standard enough for the Lalamove/Grab API geocoding service to resolve it without requiring map-pin manual drops.*
g. *Buyers will reply within the 30-minute "Customer Confirmation Loop" timeout window. If buyers consistently take hours to confirm orders, the system will experience a high rate of expired sessions.*
h. *The initial Excel document uploaded by the wholesaler contains logical, non-contradictory business rules that do not create infinite loops for the reasoning engine, so that AI can apply discounts and substitutions correctly*

© UMHackathon 2026

i. *Buyers will accept the 10-to-30 second processing lag for voice transcription (or the 90-second end-to-end processing time) without spamming the WhatsApp bot which could trigger duplicate processing.*

j. *We assume the volume of simultaneous orders for a single SME wholesaler will not exceed the throughput capacity of the in-process serial queue, so that Google Sheets write-backs do not bottleneck heavily during festive peak hours.*

4.4. ***External Dependencies:*** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks & Mitigation Strategy*
Z Ai GLM API (GLM-5.1, GLM-4.6V)	*It is the core LLM reasoning engine for customer intent extraction, multilingual speech-to-text transcription, handwritten image parsing, and natural language customer response generation.*	*High- If the system hits the GLM API rate limit or the servers go offline, the core product will completely fail and stop processing orders automatically.* *Strategy:* *Limit unnecessary API calls using caching. When the API fails, the system pauses AI processing, alerts staff, and queues incoming WhatsApp messages so the team can handle orders manually*
WhatsApp Business API (via Twilio)	*It serves as the sole inbound or outbound communication channel for buyers to send text, voice notes, and images, and receive quotes.*	*Medium- Official WhatsApp Business API approval takes time. For the MVP, we are using the free Twilio Sandbox, which has strict daily message rate limits and strict "opt-in" rules that could block messages during the live demo.* *Strategy:* *If Twilio rate limits are hit or the WhatsApp account is not*

© UMHackathon 2026

		fully approved, the team will switch to a "WhatsApp-mimicking Mock Chat UI". This custom web interface connects directly to the exact same AI backend pipeline and can still see the end-to-end workflow without relying on Twilio's live servers.
Logistics APIs	*To calculate delivery fees based on distance, weights and auto book drivers once the order is confirmed.*	*Medium- Getting logistics APIs requires official business documents. Real API responses can take more than 15s which is slow.* *Strategy:* *Built a mock API that returns the delivery quotes instantly. In real world applications, if the API fails, the system will fallback to alert the staff to book delivery manually.*
Programmable Voice API (Twilio Voice – Mocked)	*Handles live phone calls from buyers, records the audio, and sends it to Whisper to turn speech into text.*	*Low- Building and demoing a live phone routing system is technically complex and can be unstable and prone to crashing* *Strategy:* *We will directly insert a realistic "live call audio" straight into the AI pipeline to prove our AI can perfectly understand and process messy phone conversations*

5. Project Management & Team Contributions

© UMHackathon 2026

The development of SupplyLah followed an agile 10-day sprint, prioritizing a "Design-First" approach to ensure the complex multi-agent orchestration was logically sound before full-scale implementation.

5.1. *Project Timeline:*

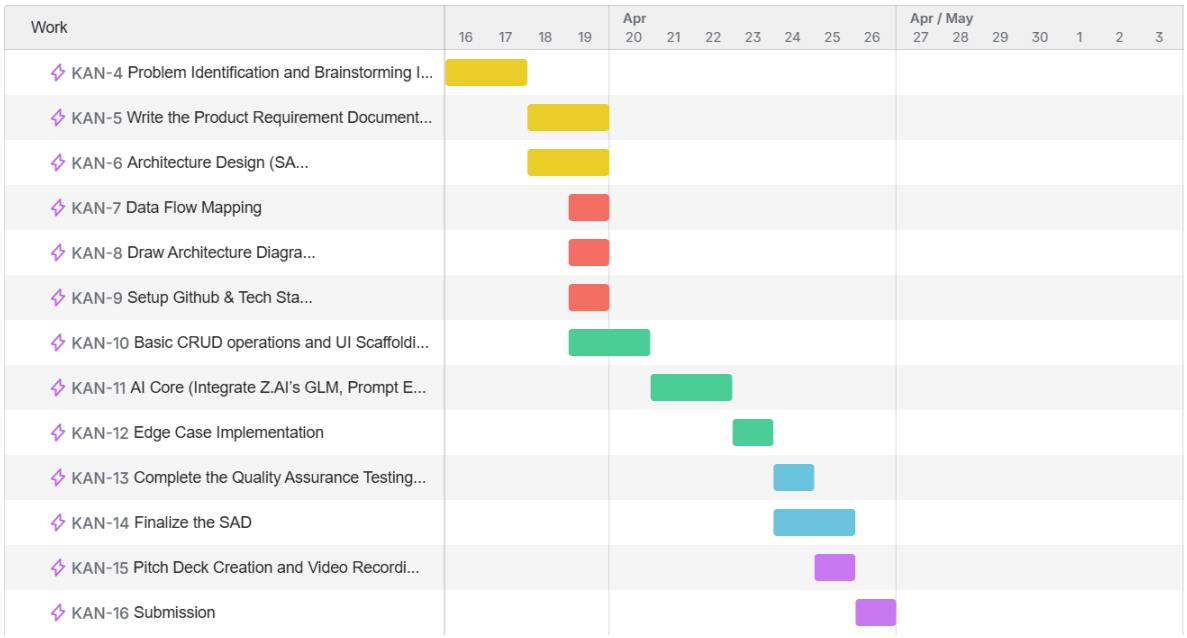
Apr 16-17:
Problem Identification and Brainstorming Ideas

Apr 18-19:
Write the Product Requirement Document (PRD)
Architecture Design (SAD)
Data Flow Mapping
Draw Architecture Diagram
Setup Github & Tech Stack

Apr 19 - Apr 23 (Development):
Apr 19-20: Basic CRUD operations and UI Scaffolding.
Apr 21-22: AI Core (Integrate Z.AI's GLM, Prompt Engineering, Multi-step Reasoning)
Apr 23: Edge Case Implementation

Apr 24 - Apr 25 (Testing & Documentation):
Complete the Quality Assurance Testing Document (QATD).
Finalize the System Analysis Document (SAD)
Pitch Deck Creation and Video Recording

Apr 26:
Submission



© UMHackathon 2026

5.2. *Team Members Role:*

Team Member	*Task*
Ooi Rui Zhe	• Responsible for orchestrator logic, multi-agent state machine, and conversation routing across different states. • Built the GLM integration, implementing the GLM-5.1 agentic loop with tool-calling, retry handling, and GOAWAY mitigation. • Designed inventory write queue logic to prevent race conditions during stock deduction. • Engineered full buyer conversation flows (substitution handling, partial stock, repeat orders, restock notifications, address validation). • Developed the Next.js Command Centre, including real-time Kanban pipeline and analytics dashboard (Revenue, AI Performance, Substitution, Products, Customers). • Built the mock-chat demo interface with SSE real-time logs, WhatsApp-style UI and demo identity switching.
Vanness Liu Chuen Wei	• Responsible for FastAPI microservices architecture, including Twilio WhatsApp webhook and mock-chat SSE endpoints. • Designed and implemented the full Supabase PostgreSQL schema (customer, order, product, logs, logistics, knowledge base). • Implemented serialized inventory write queue to prevent overselling

© UMHackathon 2026

	under concurrent requests. • Built the voice processing pipeline (S3 upload, Groq Whisper transcription, pre-signed URL flow, demo audio simulation). • Integrated Lalamove delivery API (mock) with pricing, vehicle selection, and tracking flow.
Ong Zhao Qian	• Responsible for the Product Requirements Document (PRD), defining agent modules, buyer flows, and KPIs. • Led QA testing (QATD) across multilingual scenarios (English, Bahasa Melayu, Bahasa Rojak). • Identified and documented edge cases (address loops, language misdetection, stale substitution states). • Refined prompt engineering for Intake, Inventory, and Logistics agents (multilingual extraction, substitution rules, response tone). • Validated end-to-end demo flow for hackathon presentation.

5.3. Recommendations:

- **State Management with Redis:**
  Currently, state is managed via Supabase. For high-concurrency environments (e.g., festive seasons), we recommend using Redis Cache to store active conversation states and "Negotiation Loops" for sub-millisecond latency.

- **Vector Search for Product Catalogs:**
  As merchant catalogs grow from 100 to 10,000+ items, injecting the full list into the prompt is inefficient. We recommend implementing Retrieval-Augmented Generation (RAG) using a Vector Database (like pgvector) to allow the GLM to query only relevant products.

© UMHackathon 2026

- **Distributed Task Queues (Celery/RabbitMQ):**
  To ensure reliability in logistics booking (e.g., actual Lalamove/Grab integrations), using a distributed task queue would prevent system timeouts if external APIs are slow or down.

- **Multi-regional Deployment:**
  To support Ant International's global trade vision, the architecture should be containerized using Docker/Kubernetes and deployed on AWS/Huawei Cloud in regions closest to the logistics providers to minimize latency.

- **LLM Response Caching**:
  Cache repeated inquiry responses (stock, pricing) with short TTL to reduce LLM API costs and latency at scale.

- **Circuit Breaker with Fallback LLM:**
  If the main AI model (GLM) becomes slow, fails, or goes down, the system will automatically switch to a backup AI model to continue processing requests. This ensures uninterrupted order processing during provider outages.

- **Durable Webhook Queue with AWS SQS/Kafka**:
  The current implementation uses an in-memory asyncio, queue which handles concurrency within a single process but loses queued messages on server restart. Replacing this with a durable queue ensures messages are not lost even if the server restarts, and allows the system to handle high traffic more reliably.

© UMHackathon 2026